

```

148 -- 1. Write a SQL query to find all salesman and clients located in the city of Ha Noi on a table with information: ID, Name, City and Type.
149 • SELECT c.client_number ID, c.client_name Name, c.City City, 'Client' Type
150 FROM clients c
151 WHERE c.City = 'Hanoi'
152 UNION
153 SELECT s.salesman_number ID, s.salesman_name Name, s.City City, 'Salesman' Type
154 FROM salesman s
155 WHERE s.City = 'Hanoi';

```

Result Grid   Filter Rows:  Export:  Wrap Cell Content: 

|   | ID   | Name      | City  | Type     |
|---|------|-----------|-------|----------|
| ▶ | C105 | Ho Nhu    | Hanoi | Client   |
|   | C110 | Tran Minh | Hanoi | Client   |
|   | S002 | Phat      | Hanoi | Salesman |

```

157 -- 2. Write a SQL query to create a union of two queries that shows the salesman, cities, and target_achieved of all salesmen.
158 -- Those with a target of 60 or greater will have the words 'High Achieved', while the others will have the words 'Low Achieved'.
159 • select s1.salesman_number, s1.salesman_name, s1.city, s1.target_achieved, 'High Achieved' status
160 from salesman s1
161 where s1.target_achieved >= 60
162 union
163 select s2.salesman_number, s2.salesman_name, s2.city, s2.target_achieved, 'Low Achieved' status
164 from salesman s2
165 where s2.target_achieved < 60;

```

Result Grid   Filter Rows:  Export:  Wrap Cell Content: 

|   | salesman_number | salesman_name | city        | target_achieved | status        |
|---|-----------------|---------------|-------------|-----------------|---------------|
| ▶ | S002            | Phat          | Hanoi       | 110             | High Achieved |
|   | S004            | Tien          | Dai An      | 72              | High Achieved |
|   | S001            | Huu           | Ho Chi Minh | 35              | Low Achieved  |
|   | S003            | Khoa          | Thu Dau Mot | 30              | Low Achieved  |
|   | S005            | Deb           | Thu Dau Mot | 48              | Low Achieved  |
|   | S006            | Tin           | Da Lat      | 55              | Low Achieved  |

```

167 -- 3. Write a SQL query to find those salesman and clients who have placed more than one order. Return ID, name and order by ID.
168 • select s.salesman_number ID, s.salesman_name Name
169 from salesorder so
170 join salesman s on s.salesman_number = so.salesman_number
171 group by s.salesman_number
172 having count(s.salesman_number) > 1
173 union
174 select c.client_number ID, c.client_name Name
175 from salesorder so
176 join clients c on c.client_number = so.client_number
177 group by c.client_number
178 having count(c.client_number) > 1
179 order by ID;

```

Result Grid   Filter Rows:  Export:  Wrap Cell Content: 

|   | ID   | Name     |
|---|------|----------|
| ▶ | C101 | Mai Xuan |
|   | C102 | Le Xuan  |
|   | C106 | Tran Hai |
|   | S001 | Huu      |
|   | S003 | Khoa     |
|   | S004 | Tien     |
|   | S005 | Deb      |
|   | S006 | Tin      |

```

181 -- 4. Retrieve the distinct client names who placed orders and the salesman names who processed those orders
182 • select distinct s.salesman_name Name
183 from salesorder so
184 join salesman s on s.salesman_number = so.salesman_number
185 union
186 select distinct c.client_name Name
187 from salesorder so
188 join clients c on c.client_number = so.client_number;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

| Name      |
|-----------|
| Huu       |
| Phat      |
| Khoa      |
| Tien      |
| Deb       |
| Tin       |
| Mai Xuan  |
| Le Xuan   |
| Trinh Huu |
| Tran Tuan |
| Ho Nhu    |

```

190 -- 5. Create a procedure stores get_clients() saves the all Clients in table. Then Call procedure stores.
191 delimiter //
192 • create procedure get_client()
193 begin
194     select * from clients;
195 end //
196 delimiter ;
197
198 • call get_client();

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

|   | Client_Number | Client_Name  | Address  | City        | Pincode | Province    | Amount_Paid | Amount_Due |
|---|---------------|--------------|----------|-------------|---------|-------------|-------------|------------|
| ▶ | C101          | Mai Xuan     | Phu Hoa  | Dai An      | 700001  | Binh Duong  | 10000.0000  | 5000.0000  |
|   | C102          | Le Xuan      | Phu Hoa  | Thu Dau Mot | 700051  | Binh Duong  | 18000.0000  | 3000.0000  |
|   | C103          | Trinh Huu    | Phu Loi  | Da Lat      | 700051  | Lam Dong    | 7000.0000   | 3200.0000  |
|   | C104          | Tran Tuan    | Phu Tan  | Thu Dau Mot | 700080  | Binh Duong  | 8000.0000   | 0.0000     |
|   | C105          | Ho Nhu       | Chanh My | Hanoi       | 700005  | Hanoi       | 7000.0000   | 150.0000   |
|   | C106          | Tran Hai     | Phu Hoa  | Ho Chi Minh | 700002  | Ho Chi Minh | 7000.0000   | 1300.0000  |
|   | C107          | Nguyen Thanh | Hoa Phu  | Dai An      | 700023  | Binh Duong  | 8500.0000   | 7500.0000  |
|   | C108          | Nguyen Sy    | Tan An   | Da Lat      | 700032  | Lam Dong    | 15000.0000  | 1000.0000  |
|   | C109          | Duong Thanh  | Phu Hoa  | Ho Chi Minh | 700011  | Ho Chi Minh | 12000.0000  | 8000.0000  |
|   | C110          | Tran Minh    | Phu My   | Hanoi       | 700005  | Hanoi       | 9000.0000   | 1000.0000  |

```

200    -- 6. Drop get_clients() procedure stores.
201 •   drop procedure if exists get_client;
202
203    -- 7. Create a stored procedure to retrieve all clients in a specific city
204    delimiter //
205 •   create procedure get_client_by_city
206   (
207       IN city_name varchar(30)
208   )
209   begin
210       select * from clients
211       where city = city_name;
212   end //
213   delimiter ;
214
215 •   call get_client_by_city('Hanoi');
...

```

| <div> <div>Result Grid</div> <div> <div></div> <div></div> </div> <div>Filter Rows: <input type="text"/></div> <div> <div>Export:</div> <div></div> </div> <div>Wrap Cell Content: <span>IA</span></div> </div> |               |             |          |       |         |          |             |            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------|----------|-------|---------|----------|-------------|------------|
|                                                                                                                                                                                                                 | Client_Number | Client_Name | Address  | City  | Pincode | Province | Amount_Paid | Amount_Due |
| ►                                                                                                                                                                                                               | C105          | Ho Nhu      | Chanh My | Hanoi | 700005  | Hanoi    | 7000.0000   | 150.0000   |
|                                                                                                                                                                                                                 | C110          | Tran Minh   | Phu My   | Hanoi | 700005  | Hanoi    | 9000.0000   | 1000.0000  |

```

217 -- 8. Create a stored procedure to update the delivery status for a given product number.
218 • drop procedure if exists update_delivery_status;
219
220 delimiter //
221 • create procedure update_delivery_status
222 (
223     IN in_product_number varchar(15),
224     IN in_delivery_status char(15)
225 )
226 begin
227     update salesorder so
228     set so.delivery_status = in_delivery_status
229     where so.order_number in (
230         select sod.order_number
231         from salesorderdetails sod
232         where sod.product_number = in_product_number
233     );
234 end //
235 delimiter ;
236
237 • call update_delivery_status('P1008', 'Ready to Ship');
238
239 -- 9. Create a stored procedure to retrieve the total quantity for each product.
240 • drop procedure if exists get_total_quantity_product;
241
242 delimiter //
243 • create procedure get_total_quantity_product()
244 begin
245     select p.product_number, p.product_name, sum(sod.order_quantity) total_quantity
246     from product p
247     join salesorderdetails sod using (product_number)
248     group by p.product_number;
249 end //
250 delimiter ;
251
252 • call get_total_quantity_product();

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

|   | product_number | product_name | total_quantity |
|---|----------------|--------------|----------------|
| ▶ | P1001          | TV           | 24             |
|   | P1002          | Laptop       | 43             |
|   | P1003          | AC           | 12             |
|   | P1004          | Modem        | 11             |
|   | P1005          | Pen          | 9              |
|   | P1006          | Mouse        | 21             |

```

254 -- 10. Create a stored procedure to update the remarks for a specific salesman.
255 • alter table salesman
256 add column remark varchar(25);
257
258 delimiter //
259 • create procedure update_remark_specific_salesman
260 (
261     IN in_salesman_number varchar(25),
262     IN in_salesman_remark varchar(25)
263 )
264 begin
265     update salesman
266     set remark = in_salesman_remark
267     where salesman_number = in_salesman_number;
268 end //
269 delimiter ;
270
271 • call update_remark_specific_salesman('S001', 'Good');
272
273 -- 11. Create a procedure stores find_clients() saves all of clients and can call each client by client_number.
274 delimiter //
275 • create procedure find_client
276 (
277     IN in_client_number varchar(10)
278 )
279 begin
280     select * from clients
281     where client_number = IFNULL(in_client_number, client_number);
282 end //
283 delimiter ;
284
285 • call find_client(NULL);
286 • call find_client('C101');
287 ---

```

| Result Grid   |             |         |        |                    |            |             |            |
|---------------|-------------|---------|--------|--------------------|------------|-------------|------------|
| Filter Rows:  |             | Export: |        | Wrap Cell Content: |            |             |            |
| Client_Number | Client_Name | Address | City   | Pincode            | Province   | Amount_Paid | Amount_Due |
| C101          | Mai Xuan    | Phu Hoa | Dai An | 700001             | Binh Duong | 10000.0000  | 5000.0000  |

```

288 -- 12. Create a procedure stores salary_salesman() saves all of clients (salesman_number, salesman_name, salary) having salary >15000.
289 -- Then execute the first 2 rows and the first 4 rows from the salesman table.
290 • drop procedure if exists salary_salesman;
291
292 delimiter //
293 • create procedure salary_salesman
294 (
295     IN limit_rows int
296 )
297 begin
298     select s.salesman_number, s.salesman_name, s.salary
299     from salesman s
300     where s.salary > 15000
301     limit limit_rows;
302 end //
303 delimiter ;
304
305 • call salary_salesman(4);
---
```

| salesman_number | salesman_name | salary     |
|-----------------|---------------|------------|
| S002            | Phat          | 25000.0000 |
| S003            | Khoa          | 17500.0000 |
| S004            | Tien          | 16500.0000 |
| S006            | Tin           | 20000.0000 |

```

307 -- 13. Procedure MySQL MAX() function retrieves maximum salary from MAX_SALARY of salary table.
308 • drop function if exists max_salary_salesman;
309 delimiter //
310 • create function max_salary_salesman()
311 returns decimal(15,4)
312 DETERMINISTIC
313 begin
314     declare max_salary decimal(15,4);
315     select max(salary)
316     into max_salary
317     from salesman;
318     return max_salary;
319 end //
320 delimiter ;
321
322 • select max_salary_salesman();
---
```

| max_salary_salesman() |
|-----------------------|
| 25000.0000            |

```

324 -- 14. Create a procedure stores execute finding amount of order_status by values order status of salesorder table.
325 • drop procedure if exists find_amount_order_status;
326 delimiter //
327 • create procedure find_amount_order_status
328 (
329     IN in_order_status varchar(15)
330 )
331 begin
332     select count(*) amount_order_status from salesorder
333     where order_status = in_order_status;
334 end //
335 delimiter ;
336
337 • call find_amount_order_status('Successful');

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: 

| amount_order_status |
|---------------------|
| 8                   |

```

339 -- 15. Create a stored procedure to calculate and update the discount rate for orders.
340 • drop procedure if exists update_rate_order;
341 delimiter //
342 • create procedure update_rate_order()
343 begin
344     update salesorderdetails
345     set discount_rate =
346     case
347         when order_quantity > 10 then 15
348         when order_quantity > 5 then 10 else 0
349     end;
350 end //
351 delimiter ;
352
353 • call update_rate_order();

```

```

355 -- 16. Count the number of salesman with following conditions : SALARY < 20000; SALARY > 20000; SALARY = 20000.
356 • drop procedure if exists count_the_number_of_salesman;
357 delimiter //
358 • create procedure count_the_number_of_salesman()
359 begin
360     select 'Under 20000' as Type, count(salary) count from salesman
361     where salary < 20000
362     union
363     select 'Equal 20000', count(salary) count from salesman
364     where salary = 20000
365     union
366     select 'Over 20000', count(salary) count from salesman
367     where salary > 20000;
368 end //
369 delimiter ;
370
371 • call count_the_number_of_salesman();
---
```

Result Grid  Filter Rows:  Export:  Wrap Cell Content: 

|   | Type        | count |
|---|-------------|-------|
| ▶ | Under 20000 | 4     |
|   | Equal 20000 | 1     |
|   | Over 20000  | 1     |

```

379 -- 17. Create a stored procedure to retrieve the total sales for a specific salesman
380 • drop procedure if exists get_total_sale;
381 delimiter //
382 • create procedure get_total_sale
383 (
384     IN in_salesman_number varchar(15)
385 )
386 begin
387     select s.salesman_number, s.salesman_name, count(so.order_number) total_sale
388     from salesman s
389     join salesorder so using (salesman_number)
390     where s.salesman_number = in_salesman_number
391     group by s.salesman_number, s.salesman_name;
392 end //
393 delimiter ;
394
395 • call get_total_sale('S001');
---
```

Result Grid  Filter Rows:  Export:  Wrap Cell Content: 

|   | salesman_number | salesman_name | total_sale |
|---|-----------------|---------------|------------|
| ▶ | S001            | Huu           | 3          |

```

397 -- 18. Create a trigger before_total_quantity_update to update total quantity of product when Quantity_On_Hand and Quantity_sell change values.
398 -- Then Update total quantity when Product P1004 have Quantity_On_Hand = 30, quantity_sell =35.
399 • alter table product
400 add column total_quantity int;
401
402 • update product
403 set total_quantity = quantity_on_hand + quantity_sell;
404
405 delimiter //
406 • create trigger before_total_quantity_update
407 before update on product
408 for each row
409 begin
410     set new.total_quantity = new.quantity_on_hand + new.quantity_sell;
411 end //
412 delimiter ;
413
414 • update product
415 set quantity_on_hand = 30, quantity_sell = 35
416 where product_number = 'P1004';
417
418 -- 19. Create a trigger before_remark_salesman_update to update Percentage of per_remarks in a salesman
419 -- table (will be stored in PER_MARKS column) : per_remarks = target_achieved*100/sales_target.
420 • alter table salesman
421 add column per_remarks decimal(5,2);
422
423 delimiter //
424 • create trigger before_remark_salesman_update
425 before update on salesman
426 for each row
427 begin
428     set new.per_remarks = new.target_achieved * 100 / new.sales_target;
429 end //
430 delimiter ;
431
432 • update salesman
433 set per_remarks = target_achieved * 100 / sales_target
434 where sales_target <> 0;
435
436 -- 20. Create a trigger before_product_insert to insert a product in product table.
437 delimiter //
438 • create trigger before_product_insert
439 before insert on product
440 for each row
441 begin
442     if new.sell_price < new.cost_price then
443         set new.sell_price = new.cost_price;
444         -- Sell_price must be >= cost_price
445     end if;
446 end //
447 delimiter ;

```

```

449 -- 21. Create a trigger to update the delivery status to "Delivered" when an order is marked as "Shipped".
450 delimiter //
451 • create trigger after_order_shipped
452 after update on salesorder
453 for each row
454 begin
455     if new.order_status = 'Ready to ship' then
456         -- Don't have "shipped", but have "Ready to ship"
457         update salesorder
458             set delivery_status = 'Delivered'
459             where new.order_number = order_number;
460     end if;
461 end //
462 delimiter ;

464 -- 22. Create a trigger to update the remarks for all salesmen to "Good" when a new salesman is inserted
465 delimiter //
466 • create trigger after_salesman_insert
467 after insert on salesman
468 for each row
469 begin
470     update salesman
471         set remark = 'Good'
472         where new.salesman_number = salesman_number;
473 end //
474 delimiter ;

476 -- 23. Create a trigger to enforce that the first digit of the pin code in the "Clients" table must be 7.
477 delimiter //
478 • create trigger before_client_insert |
479 before insert on clients
480 for each row
481 begin
482     declare tmp_pincode int;
483     set tmp_pincode = pincode;
484     while tmp_pincode > 10 do
485         set tmp_pincode = floor(tmp_pincode / 10);
486     end while;
487     if (tmp_pincode <> 7) then
488         signal sqlstate '45000'
489         set message_text = 'Pincode must start with 7';
490     end if;
491 end //
492 delimiter ;

```

```

494 • -- 24. Create a trigger to update the city for a specific client to "Unknown" when the client is deleted.
495 alter table clients
496 add column deleted_status boolean default false;
497
498 delimiter //
499 • create trigger before_client_delete
500 before delete on clients
501 for each row
502 begin
503     update clients
504     set city = 'Unknown', deleted_status = true
505     where old.client_number = client_number and deleted_status = false;
506 end //
507 delimiter ;
509 -- 25. Create a trigger to update the delivery status to "Cancelled" for corresponding order details when an order is cancelled
510 delimiter //
511 • create trigger after_order_cancelled
512 after update on salesorder
513 for each row
514 begin
515     if new.order_status = 'Cancelled' then
516         update salesorder
517         set delivery_status = 'Cancelled'
518         where new.order_number = order_number;
519     end if;
520 end //
521 delimiter ;
523 -- 26. Create a trigger to update the delivery status to "Pending" for a specific order when an order is inserted
524 delimiter //
525 • create trigger after_order_insert
526 after insert on salesorder
527 for each row
528 begin
529     update salesorder
530     set new.delivery_status = 'Pending';
531 end //
532 delimiter ;
533 • drop trigger after_order_insert;
535 -- 27. Create a trigger before_remark_salesman_update to update Percentage of per_remarks in a salesman table (will be stored in PER_MARKS column)
536 -- If per_remarks >= 75%, his remarks is 'Good'. If 50% <= per_remarks < 75%, he is 'Average'. If per_remarks < 50%, he is 'Poor'.
537 delimiter //
538 create trigger before_remark_salesman_update
539 before update on salesman
540 for each row
541 begin
542     set new.per_remarks = new.target_achieved * 100 / new.sales_target;
543     update salesman
544     set remark = case
545         when new.per_marks >= 75 then 'Good'
546         when new.per_marks >= 50 then 'Average' else 'Poor'
547     end;
548 end
549 delimiter ;

```